

# AEGISPAY

## The Trust & Growth Layer for Agentic Commerce

A production-ready architecture for letting AI agents grow and sell — **without ever giving the AI control of money.**

**AI may propose a financial action. Only the deterministic AegisPay control plane may authorize and execute it. No AI can move money on its own.**

### 1. What is AegisPay?

AegisPay sits between AI agents (like a shopping assistant) and payment providers (like Razorpay). It has three jobs, written simply:

| Job     | Plain-English meaning                                                                                        |
|---------|--------------------------------------------------------------------------------------------------------------|
| GROW    | Use AI to help a merchant sell more (suggest cross-sells, upsells, bundles, campaigns).                      |
| SELL    | Let an AI buyer find a product, build a cart, and pay end-to-end.                                            |
| PROTECT | Make every AI-driven money step explainable, limited, approved, recorded, and safe to recover from failures. |

The reason AegisPay is trusted is that **PROTECT is baked into the architecture** — GROW and SELL work because money can never be moved by the AI alone.

## 2. The architecture at a glance

This is the whole system. Read it from left to right: a user asks an AI buyer, the agent talks to AegisPay, AegisPay checks everything, and only then does it ask Razorpay to move money. Razorpay replies through verified webhooks, and AegisPay records every step.

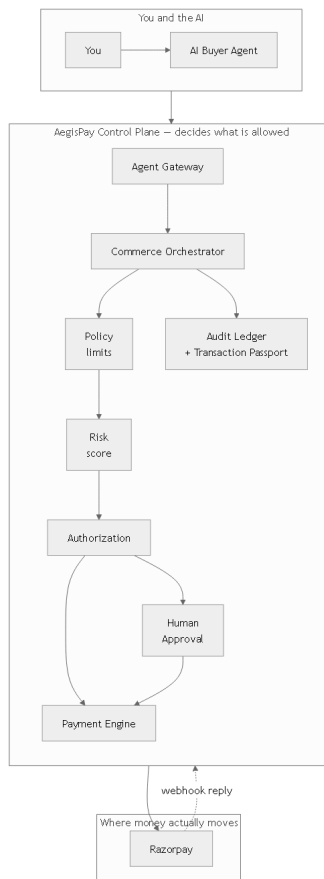


Figure 1 — AegisPay high-level architecture

### Why this is hard to reject:

striped **No secret** is given to the AI — no payment keys, no database passwords.

striped **No payment code** is reachable by the AI. The AI only *requests*; AegisPay decides.

striped **Every action is logged** in a tamper-proof audit ledger and a Transaction Passport.

striped **Failures are safe** — if Razorpay is slow, AegisPay waits and checks, it never guesses and never double-charges.

## 3. The one rule, every time

Every financial action follows this exact path. There is no shortcut, and no AI can skip a step.

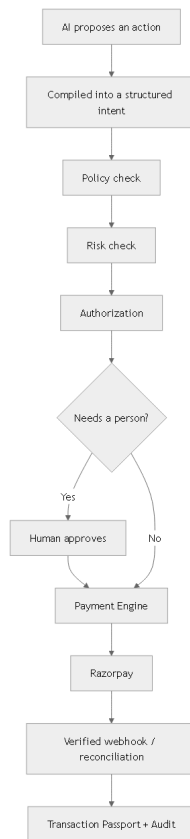


Figure 2 — The decision path every money action must pass

The important bit: the AI produces the *idea* (an intent). AegisPay converts that idea into a checked, structured request and decides yes/no using rules, not guesses.

## 4. Step-by-step: a real purchase

Here is the exact journey for a person asking an AI to buy running shoes under Rs 4,000.

- 1 The user tells the AI buyer: “Find running shoes under Rs 4,000.”
- 2 The AI searches the merchant's catalog and recommends shoes.
- 3 The AI builds a **cart**. AegisPay writes the prices itself — the AI cannot set a price.
- 4 AegisPay turns the AI's plan into a structured **intent**.
- 5 AegisPay checks **policy**: is this amount allowed, is this category allowed, is it within today's limit?
- 6 AegisPay checks **risk**: is this a normal purchase or risky?
- 7 AegisPay **authorizes**: if low risk it approves; if high risk it asks a human first.
- 8 Only then does AegisPay ask **Razorpay** to create the order and take payment.
- 9 Razorpay confirms through a **signed webhook** (and reconciliation if needed).
- 10 AegisPay writes the **Transaction Passport** and the **audit record**.
- 11 The user gets a confirmation with the full evidence.

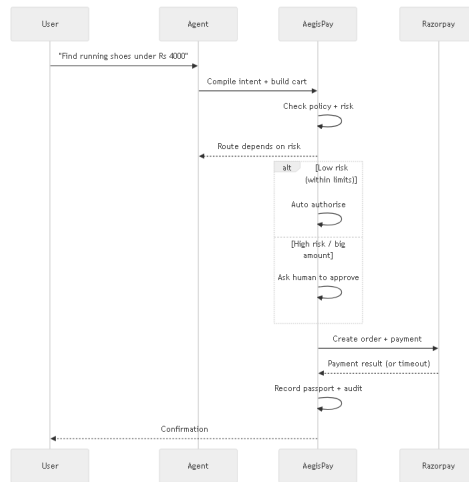


Figure 3 — Automated purchase workflow (including the human-approval branch)

## 5. The safety layers

AegisPay does not give the AI freedom. It gives the AI freedom *inside limits*, and it escalates when it should.

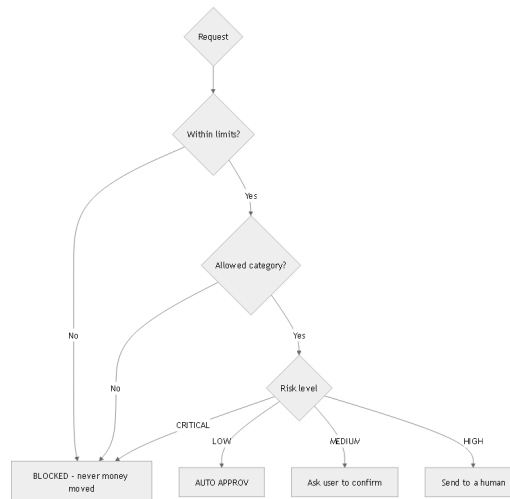


Figure 4 — Decision layers: limits, then risk, then people

| Outcome            | What it means                                                                |
|--------------------|------------------------------------------------------------------------------|
| AUTO APPROVE       | Low-risk, within all limits. No human needed.                                |
| ASK USER (step-up) | Medium risk. The user must confirm that one action.                          |
| SEND TO A HUMAN    | High risk. A person reviews the whys and decides.                            |
| BLOCK              | Outside limits or critical risk. Money never moves; the attempt is recorded. |

## 6. What happens when things go wrong

Payment systems are judged by how they fail. AegisPay fails safely.

### Scenario — Razorpay is slow and does not answer in time:

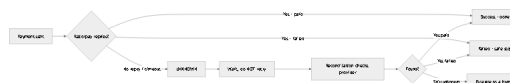


Figure 5 — Unknown payment is resolved by checking, never by guessing

The key: **AegisPay never blindly retries.** If the first attempt actually worked, retrying would double-charge the customer. So AegisPay marks the payment UNKNOWN, then quietly asks Razorpay what really happened, and finishes the job safely.

### Scenario — the same request is sent twice by accident:



Figure 6 — Duplicate request guard (idempotency)

Every money request gets a unique key. A repeated request returns the same stored answer and is never charged twice.

## 7. Many merchants, no leaks

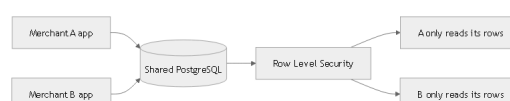


Figure 7 — Tenant isolation

Thousands of merchants share one database, but each can only see its own rows. AegisPay gives every table a **tenant id** and enforces it at the database level using **Row-Level Security**, plus checks in the code. Merchant A can never read Merchant B's data.

## 8. Proof you can actually use

Every decision leaves a tamper-proof trail. Events are linked like a chain, so if anyone changes an old record, the chain breaks and the system flags it.

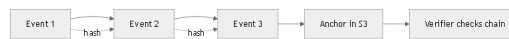


Figure 8 — The audit chain

**Transaction Passport.** For any transaction you get one clean, human-readable page showing: what was bought, who wanted it, which agent, which policy version, the risk, the approval, the Razorpay order id, and a verified integrity stamp. That is answer to “why did this happen?”

| Passport field       | Purpose                                                |
|----------------------|--------------------------------------------------------|
| Intent / cart hashes | Proves the approved cart is exactly what was paid for. |
| Policy version       | Tells you exactly which rules were applied.            |
| Risk score           | Shows how risky this was and why.                      |
| Authorization hash   | Proves an authorized person/agent backed it.           |
| Human approval       | Shows whether a person approved it.                    |
| Provider order id    | Links to the real Razorpay record.                     |
| Audit integrity      | Verifies nothing was altered afterwards.               |

## 9. The technology (kept simple and reliable)

AegisPay is **all Python/FastAPI** — one language, one toolchain, easy for any team to read and run. The AI part is a *separate* FastAPI service that is not allowed to touch money. AegisPay uses boring, dependable technology because money needs boring.

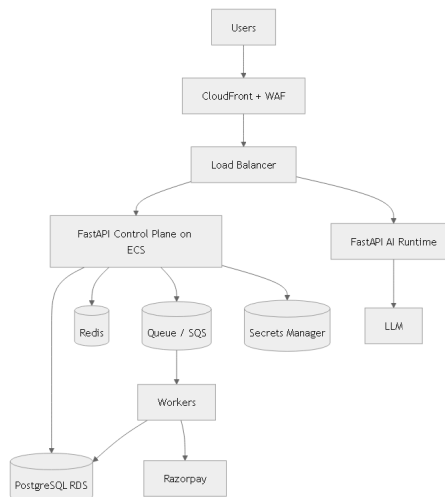


Figure 9 — AWS deployment

| Layer         | Choice                      | Why                                                                                          |
|---------------|-----------------------------|----------------------------------------------------------------------------------------------|
| Core          | Python / FastAPI            | One language, fast to build, real-time async, strong typed validation. Safe for money logic. |
| AI runtime    | Python / FastAPI (separate) | Same language, but a separate service that never touches money — keeps the AI safe.          |
| Database      | PostgreSQL                  | Correct, auditable, per-tenant security, easy backup.                                        |
| Cache / locks | Redis                       | Fast caching and rate limits; never the source of truth.                                     |
| Queue         | SQS                         | Reliable background work (webhooks, reconciliation, reports).                                |

|          |                          |                                                          |
|----------|--------------------------|----------------------------------------------------------|
| Cloud    | AWS ECS (not Kubernetes) | Simple, reliable, less to go wrong at this scale.        |
| Frontend | Next.js + TypeScript     | Merchant dashboard, approval inbox, catalogs, analytics. |

## 10. Why this is a good architecture

Anyone can draw a clever diagram. What makes this one good is that **safety is built in**, not bolted on. Here is what that means:

| Quality                      | How this architecture provides it                                                                                  |
|------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Safe by default              | The AI can never move money. It has no payment keys and no payment tool — only a request the system always checks. |
| No double charges            | Idempotency keys + never-retry-unknown + reconciliation make duplicate payments impossible.                        |
| No way around the rules      | Every money action must pass policy, risk and authorization. There is no code path that skips them.                |
| A person is always in charge | High-risk actions go to a human; limits are set by the merchant, not by the AI.                                    |
| You can prove things         | The audit chain and Transaction Passport answer who, what, when and why with evidence.                             |
| Grows safely                 | Autonomy is dialed up gradually, and every higher level gets more scrutiny — never less.                           |
| Ready for the future         | MCP, A2A and new protocols plug in as adapters without touching the safe money core.                               |
| Simple to run                | One main service, one database, one queue. Fewer moving parts means fewer things to break.                         |

**In one line: AegisPay lets AI grow and sell — and is the amount of trust you would actually hand to an AI.**